

Evaluating Enterprise Agents From the Traces They Leave

Lorenzo Antonelli[✉]
antonelli@diag.uniroma1.it
Sapienza University of Rome
Syllotips
Rome, Italy

Andrea Rossi
andrea.rossi@syllotips.com
Syllotips
Rome, Italy

Leonardo Martini
leonardo.martini@syllotips.com
Syllotips
Rome, Italy

Fabrizio Silvestri
fsilvestri@diag.uniroma1.it
Sapienza University of Rome
Rome, Italy

Gabriele Tolomei
tolomei@di.uniroma1.it
Sapienza University of Rome
Rome, Italy

Abstract

The adoption of AI agents in enterprise environments is hindered by unreliability, motivating researchers to diagnose and classify failures, benchmark the behavior of agents, and test their capabilities. Amid a fast-growing body of work and tools, enterprise practitioners face two gaps: on the one hand they lack a bird's eye view of the research field that is organized around their practical questions. On the other hand, enterprise agents typically serve narrow processes and are often third-party or low-code, scenarios where traditional evaluation frameworks that rely on their own instrumentation fall short. In this paper we address both gaps. First, we organize 77 papers into a taxonomy around five core questions: what is under test, how a verdict is produced, why and where agents fail, whether they are safe and compliant, and whether the evaluation is trustworthy. The result is a map that connects the main questions an evaluator may ask with the corresponding works that answer them. Second, we introduce SYLLO-EVAL, an open-source, extensible, post hoc evaluation framework. SYLLO-EVAL calls the agent under test as a black box, and retrieves its traces from the existing observability platform without additional instrumentation. A metric planner binds each metric to the spans it targets, so judge components see only what they assess, reducing their load. We use SYLLO-EVAL on **N** agents and **M** datasets, finding that it agrees with human judgments ($\rho = . . .$), separates agents of different quality, and remains stable as traces grow in size, while whole-trace judges degrade. Together, these contributions help practitioners navigate the field and evaluate agents they cannot modify.

CCS Concepts

• **Do Not Use This Code** → **Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

Keywords

AI Agent, LLM, Evaluation, Benchmark, Taxonomy, Framework

ACM Reference Format:

Lorenzo Antonelli, Andrea Rossi, Leonardo Martini, Fabrizio Silvestri, and Gabriele Tolomei. 2018. Evaluating Enterprise Agents From the Traces They Leave. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 13 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

As the capabilities of large language models (LLMs) advance, AI agents are taking on increasingly complex tasks, ranging from Retrieval-Augmented Generation (RAG) [45] and tool use [99] to deep research [31, 68], software engineering [34], and multi-step enterprise workflows [21, 29, 96].

Capability, however, does not necessarily imply reliability, especially in the inherent ambiguity of real-world scenarios. On τ -bench, a benchmark based on real-world customer service domains, even state-of-the-art agents solve fewer than half of the tasks, and their probability of succeeding on a retail task in all of eight trials falls below 25% [98]. Practitioners name reliability as their top challenge in development, and 74% of production agents are still evaluated primarily manually [70]. Unreliability heavily affects the adoption of agentic AI in enterprise, with Gartner predicting the cancellation of more than 40% of agentic projects by 2027 [23], and MIT finding that only 5% of custom AI tools reach production [12].

The study of why agents fail and how to catch such failures has therefore become an increasingly active research field, with the urgency of the problem leading to quick and at times fragmented growth. Researchers have systematized agentic issues into several comprehensive *failure taxonomies*, such as MAST [11] and TRAIL [20], finding that failures often originate in intermediate steps (e.g., flawed plans, misused tools, or missing verification) that the final output does not reveal [113]. In the meantime, numerous methods of *failure attribution* have been proposed to identify the agent and the step responsible for each failure [6, 36, 50, 93, 105, 107]. The problem is far from solved: on Who&When, the *de facto* standard benchmark, even the best-performing methods only attribute about 30–55% of failures to the correct steps (depending on the dataset partition) [5, 91, 94]. Researchers have also published a flurry of agentic benchmarks and datasets, measuring agent capabilities over multiple domains and tasks [37]; and, most crucially,

they have proposed novel *evaluation frameworks* that can iterate over the available examples and compute aggregated metrics in a way that is reusable across agents and datasets [65, 101].

Despite such growth, practitioners who evaluate agents, especially in enterprise environments, still face two large gaps: a conceptual one, and a practical one.

The conceptual gap. With such a large and fragmented body of research, a structured guide is sorely needed to direct practitioners' study. What is missing is an *actionable* map from the questions an evaluator may have about their agent or methodology to the existing methods that answer them. To address this gap, we propose a question-driven taxonomy of agentic evaluation methodologies. We categorize the literature around five core questions; the question-based structure allows a very broad scope while remaining natural to consult for practitioners. Unlike surveys organized by capability, domain, or evaluation process [27, 65, 101, 112], our taxonomy spans from benchmarks to LLM- and agent-based judges, failure attribution, safety, and meta-evaluation. This makes it, to the best of our knowledge, the first wide-angle taxonomy on agentic evaluation.

The practical gap. In enterprise environments, an evaluation framework should meet three key requirements: (i) it should be *task-specific*, as enterprise agents serve narrow business processes and should be measured against their own domain-specific principles and criteria [55]; (ii) it should be *trace-grounded*, accessing the detailed records of the agent's execution in order to support metrics for specific actions and components [11, 20]; and (iii) it should be *non-intrusive*, as agents are often built by separate teams or assembled on low-code platforms (Microsoft Copilot Studio [63], Gemini Enterprise Agent Platform [25], etc.), and only their traces from the organization's observability stack may be accessible to evaluators. We argue that, as of today, the available evaluation frameworks do not fully meet such requirements: most frameworks score outputs rather than executions, typically through a single LLM judgment [53, 109] that ignores intermediate steps and degrades as inputs grow [20, 51, 60]; popular frameworks such as RAGAS [22] and ARES [79] were designed specifically for RAG pipelines; and even frameworks that evaluate full trajectories, such as DeepEval [18], require instrumenting the agent or routing its telemetry to the framework's own platform, while the evaluators bundled with observability platforms [3, 41, 43, 64] only see the traces stored in their own backend. To address this gap we introduce SYLLO-EVAL, a novel post-hoc evaluation framework built around the three aforementioned requirements. SYLLO-EVAL is *task-specific*: each sample carries its own ground truth, and metrics combine deterministic reference-based measures (e.g., precision, recall, NDCG@10 over relevance labels) with LLM judges easily extendable via rubrics. It is *trace-grounded*: adapters normalize the ingested spans into pre-defined typed observations, and a metric planner splits each evaluation into units bound to a span, a group of spans, or the whole trace. In this way, the pipeline for each metric only sees what it targets, down to a single retrieved item or claim. It is *non-intrusive*: it calls the agent as a black box and collects its trace from the observability platform that already records it, without any

need for custom instrumentation. We make SYLLO-EVAL publicly available at our GitHub repository¹.

Our contribution is three-fold:

- **The first wide-angle taxonomy on agentic evaluation:** a comprehensive categorization of 77 papers, organized around the five questions an evaluator asks about an agent: what is under test (Q1), how a verdict is produced (Q2), why and where it fails (Q3), whether it is safe and compliant (Q4), and whether the evaluation can be trusted (Q5). It is the first taxonomy unifying under a natural and actionable structure benchmarking, AI judges, failure attribution, safety, and meta-evaluation.
- **SYLLO-EVAL, a novel enterprise-oriented agentic evaluation framework:** our framework is customizable to compute task- or domain-specific metrics; is natively compatible with standard OpenTelemetry trace formats [2, 69]; and collects traces directly from the observability stacks, thus requiring no additional instrumentation. It provides built-in planning, retrieval, and answer metrics, while also offering extension points for new metrics, agent connectors, and trace collectors.
- **An extensive evaluation on N agents and M datasets**, showing that SYLLO-EVAL agrees with human judgments ($\rho = \dots$), separates agents of different quality, and **remains stable as traces grow, whereas single-pass judges degrade**.

The paper is organized as follows: Section 2 discusses related works; Section 3 discusses our taxonomy in detail; Section 4 presents our SYLLO-EVAL framework; Section 5 reports our experimental results and observations; and Section 6 provides concluding remarks.

2 Related Works

Two lines of work are closest to ours: (i) taxonomies on agent evaluation and its sub-fields; and (ii) full-fledged agent evaluation frameworks, providing ways to run AI agents over datasets, collect their outcomes, and compute aggregated metrics.

2.1 Agent Evaluation Taxonomies

As mentioned in Section 1, several agent evaluation taxonomies have already been published, typically organizing existing works by capabilities, benchmarks, and frameworks [101]; by evaluation objectives and process [65]; by what and how to evaluate in multi-turn conversations [27]; or by environments and capabilities [112]. In contrast we follow a question-driven approach, systematizing works around five key questions that evaluators may ask, each with further sub-families. This makes our taxonomy a very natural guide to explore the research field.

Furthermore, our taxonomy sets out to offer a bird's eye view on the whole research field, while pre-existing ones generally dive deeper into just one or a few specific verticals. In this regard, the two works closest to us are the surveys by Mohammadi et al. [65] and by Zhu et al. [112]. However, Mohammadi et al. [65] extensively discuss safety, compliance and enterprise-specific challenges, but cover AI judges and how verdict generation only briefly, and do not address reliability as a property of the evaluation method.

¹<https://github.com/SylloTips/syllo-eval>

Analogously, Zhu et al. [112] focus on benchmarks to guide their selection, but only refers to AI judges and safety as future trends. Neither works cover failure diagnosis (e.g., failure taxonomies or failure attribution) meta-evaluation, or construct validity.

To the best of our knowledge, our taxonomy is the first to encompass all the aforementioned topics in a single high-level view, making it particularly useful to practitioners approaching the field.

2.2 Agent Evaluation Frameworks

Agent evaluation frameworks are typically defined by the type of input they rely on: e.g., the agent’s final answer [53]; (query, context, answer) RAG triples [22, 79]; trajectories as lists of chat messages [39, 40]; or traces captured by the framework’s own instrumentation or stored in its own backend [18]. SYLLO-EVAL works on traces as well, but *post hoc*, collecting them directly from the organization’s observability platform. This design choice is tailored for enterprise environments, as it makes SYLLO-EVAL immediately applicable even to third-party or low-code agents that cannot be instrumented directly, while its main requirement, an observability stack [3, 41, 43], is already standard practice in enterprises [38].

We discuss related agent evaluation frameworks in increasing order of similarity to ours:

- **RAG-based frameworks**, such as RAGAS [22] and ARES [79], compute metrics like context relevance, answer relevance, and faithfulness. They do not support full execution traces: ARES additionally requires per-domain fine-tuned judges and a human-annotated validation set. RAGAS’ recently added agentic metrics operate on flat message lists that discard span hierarchies and metadata [77].
- **Trace-level methods**, such as AgentCompass [36], Agent GPA [33], and the framework of Madvil et al. [60], share our trace-level view, clustering errors or computing assessments on trace subsets (spans or group of spans) as well; however, they aim primarily at detecting and localizing errors, placing them closer to failure diagnosis (Q3) than to evaluation frameworks.
- **DeepEval** [18] is the closest to ours: it runs agents over datasets and aggregates span-level and trajectory metrics. However, outside its hosted platform, DeepEval requires its own tracing hooks in the agent’s code (e.g., the `@observe` decorator) to compute trace-based metrics. SYLLO-EVAL instead obtains traces *post hoc*, from the organization’s observability platform. This makes it more appealing when an observability stack is already in place but the agent’s code is not accessible, a common scenario in enterprises. The two also differ in how they judge traces: while DeepEval judges read the whole trace, risking lost-in-the-middle effects [51], SYLLO-EVAL binds metrics to span subsets so each judge only receives the observations it targets, reducing its context load.

With respect to the three requirements in Section 1, each of these frameworks relaxes at least one; while SYLLO-EVAL meets all three on top of the observability platform already in place.

3 Agent Evaluation Taxonomy

As mentioned in Sections 1 and 2, our taxonomy adopts a question-driven approach: papers are organized into families based on the

research question they address, resulting in an *actionable* guide to navigate the research area.

3.1 Selection Methodology

We built the taxonomy bottom-up. We first collected peer-reviewed papers and preprints on agent evaluation published until August 2026. We then excluded works that assess LLMs directly rather than AI agents (e.g., reports of an LLM’s performance on a set of tasks), as well as papers whose main contribution is a dataset. We also retained works that introduce evaluation or diagnostic methodologies designed for LLMs or RAG pipelines when they apply to agents. This led to our final selection of 77 papers. Most of the selected works appeared in 2025 or 2026, further evidence of the rapid recent growth of the field.

For each each paper we identified the key questions it addresses, and progressively merged similar questions until we obtained the five core families of Figure 1. Within each family, we then defined subfamilies based on the setting and use cases it supports (e.g., digital, enterprise, or human-in-the-loop settings) or on the approach they take.

3.2 Families and Subfamilies

Figure 1 displays the overall taxonomy. The five final questions result in five families that cover the full spectrum of agent evaluation: Q1 asks *what* is tested; Q2, how the outcome becomes a verdict; Q3, what a failure reveals about the agent; Q4, which behaviors are unsafe or non-compliant, regardless of the outcome; and Q5, whether the whole chain can be trusted.

Q1: What aspect of agent performance is under test? Works in this family propose evaluation or benchmarking methodologies for specific tasks and environments, or under specific success criteria; in short, they define what the agent is evaluated on. The family is further split into subfamilies based on the setting in which the target agents operate or are evaluated: *digital settings*, such as websites, operating systems, and code or API ecosystems [34, 88, 95, 111], individually or in cross-environment suites [52, 62]; *enterprise settings*, where agents execute business processes on platforms such as ServiceNow or Salesforce [21, 29] or in organization-like sandboxes [89]; *human-in-the-loop settings*, where the agent interacts with simulated users or colleagues [7, 96, 98]; *planning and long-horizon settings*, where agents compose complex plans with many dependent steps [9, 24]; and *adaptation settings*, which test whether the agent improves over a sequence of related tasks [28]. Many of the benchmarks in this family focus on specific *capabilities* of the agent (e.g., planning, tool use) or on task completion in realistic scenarios [96]. They often verify success programmatically, by comparing the final state of the environment (e.g., the database after the agent’s run) with an expected state [88, 98], or, more rarely, by checking partial progress through intermediate subgoals or checkpoints [59, 96].

Q2: How is a verdict produced? This family includes works that discuss ways to evaluate agents by converting execution outputs, which are often open-ended (e.g., a free-text answer or a plan), into evaluation scores, i.e., verdicts. Our evaluation framework SYLLO-EVAL belongs to this family. The family is divided into subfamilies

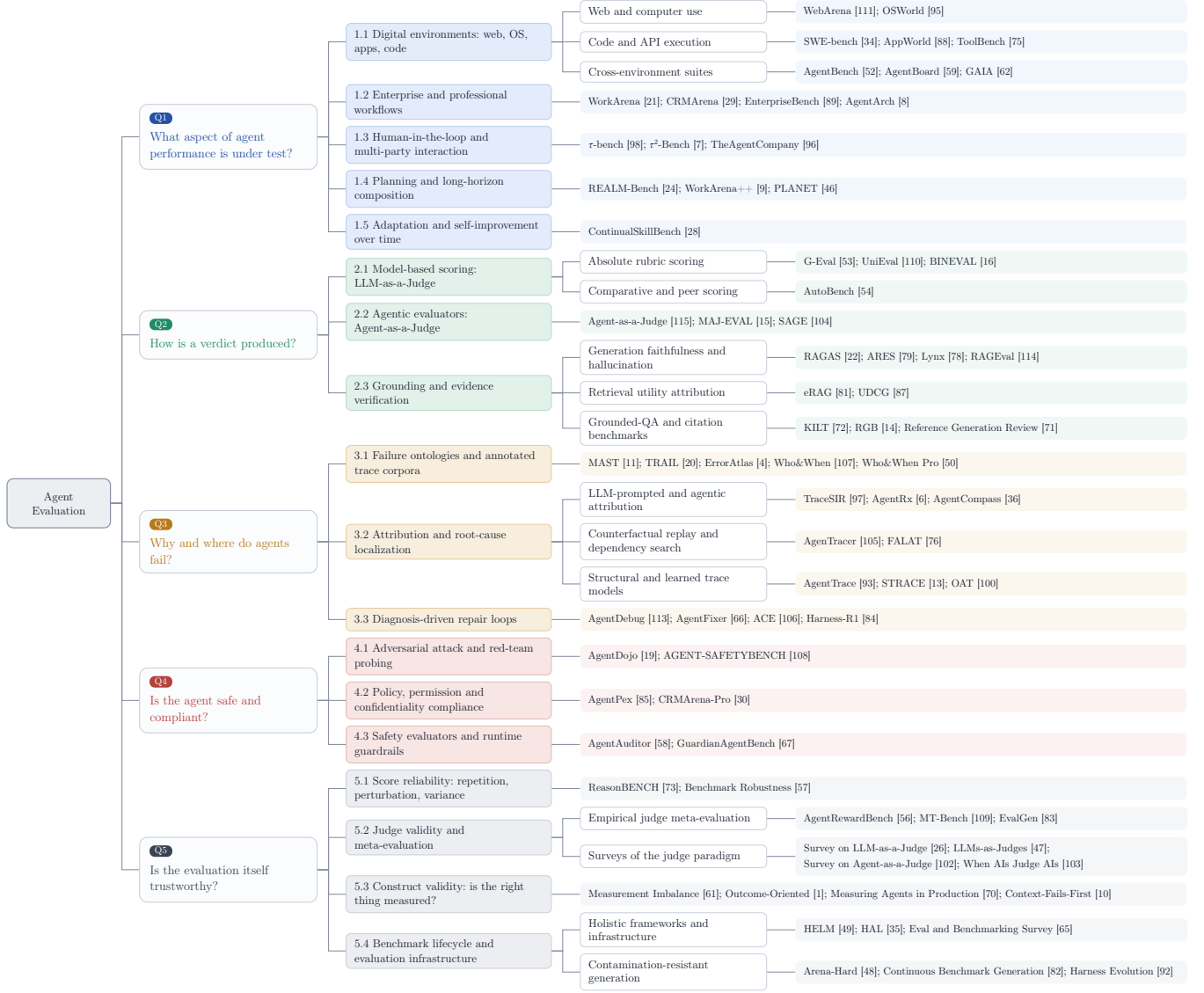


Figure 1: Question-driven taxonomy of agent evaluation. The 77 papers are organized into five families, one per core question (Q1–Q5), each divided into subfamilies and, where needed, into finer categories.

based on the nature of the component or framework that yields the verdict. *LLM-as-a-Judge methods* simply pass the output to an LLM, which generates the verdict. They may use rubrics [53], decompose the process into binary questions to increase interpretability [16, 110], or work comparatively, with models assessing one another [54]. *Agent-as-a-Judge methods* extend this paradigm by using full-fledged agents instead of a simple LLM call. Agentic judges may inspect the intermediate artifacts of the evaluated run [115], debate among multiple evaluator personas [15], or simulate a user’s emotional responses over a conversation [104]. Finally, *grounding methods* compare the output against an external source of truth, such as the retrieved context or cited references [22, 78, 79]. They may estimate how useful each retrieved document is to the generator [81, 87], or benchmark how systems answer and cite from

retrieved sources [14, 71, 72]. Many of the works in this subfamily were not specifically designed for agents, but their methodologies can also be applied to agents in evaluation frameworks.

Q3: Why and where do agents fail? This is the most recent family of the taxonomy, and it follows a natural progression: while a verdict may say whether the agent failed, it does not say why or where. That is the focus of studies of agent failures. *Failure ontologies* organize what can go wrong in single- and multi-agent systems [11, 20], and are complemented by corpora of traces annotated with the failing agent and step [50, 107]. *Failure attribution methods* localize the root cause of a failure in the trace: they may do so in a variety of ways, e.g., by prompting LLMs or agents to read the trajectory [6, 36, 97], replaying counterfactual variants of the

trace [105], searching the dependencies between its steps [76], analyzing the causal structure of the trace itself [13, 93], or modeling the dynamics of successful executions [100]. *Repair loop methods* turn diagnoses into improvements. They may rely on corrective feedback or fix recommendations [66, 113], on a context that evolves across executions [106], or on edits to the agent’s harness [84].

Q4: Is the agent safe and compliant? This family includes works that assess how agents comply with security requirements, domain policies, or access-control rules, regardless of whether the end-to-end task succeeds or fails. *Adversarial benchmarks* assess the agent before deployment by injecting malicious instructions into the data returned by its tools [19] or by exposing it to unsafe scenarios [108]. *Compliance evaluation* checks whether the agent respects explicit requirements, such as behavioral rules stated in its prompts [85] or the confidentiality of business data [30]. Finally, *safety evaluators* judge recorded interactions [58], while *runtime guardrails* move the assessment to the execution time of the agent [67].

Q5: Is the evaluation itself trustworthy? This family includes works questioning the reliability of the evaluation process itself. Its subfamilies examine each aspect of the process, from scores to benchmarks. More specifically, *score reliability studies* investigate the stability of the obtained scores across repeated runs and perturbations [57, 73]. *Judge validity studies* assess the judges used to obtain such scores, usually by measuring their agreement with human judgments [56, 109]. *Construct validity studies* investigate whether the right thing is being measured at all [61, 70]. Finally, *lifecycle and infrastructure studies* cover how evaluations are run and kept up to date, including harnesses [35] and contamination-resistant benchmark generation [48].

4 The SYLLO-EVAL Framework

The SYLLO-EVAL framework can be used to evaluate an agent on a dataset through the trace of each execution, i.e., the record of every step the agent performed. Each metric in the framework is defined over the portion of the trace that it assesses: a single span, a group of spans, or the entire trace. Retrieval, planning and replanning decisions, tool calls, and the final answer are thus each evaluated at their own level of granularity.

The trace is obtained post hoc from the organization’s observability platform (e.g., LangSmith [41] or Arize Phoenix [3]), without instrumenting the agent. This design follows requirement (iii) of Section 1, and derives directly from a constraint often found in enterprise scenarios: since agents are frequently developed and evaluated by different teams, deployed as a closed internal service, or supplied by third-party companies, evaluators are rarely allowed to directly instrument the agents under test. The only artifact that is consistently available is the execution trace, typically collected and stored in an observability platform. SYLLO-EVAL relies on that trace alone and follows the OpenTelemetry standard [2, 69] to support as many providers as possible out of the box. Unlike the message lists on which several frameworks operate (Section 2.2), spans also retain execution metadata, such as latency, token usage, cost, and the parallel execution of steps, making them a rich source of information for evaluation.

4.1 Evaluation Runs and Pipelines

Evaluation datasets are imported from JSON files where each sample specifies a test case and, optionally, ground-truths like the expected answer, the expected plan, and the IDs of the relevant documents and snippets that the agent should retrieve. Each ground-truth is stored by kind, and each metric declares the kind it uses, if any.

To execute an *evaluation run* the evaluator selects a set of built-in or custom metrics, a version of an agent, and a dataset. Agents are identified by name and version tag, so that different versions of the same agent can be evaluated on the same samples. The evaluator can also provide an agent caller, i.e., a component that runs the agent on a specific test case so that a new trace can be collected; and a client for the observability platform, if not already supported. At creation, a run stores a snapshot of the samples to evaluate, the selected metrics, and its configuration (e.g., concurrency, timeouts).

The evaluation run executes an *evaluation pipeline* on each sample of the dataset through a configurable number of concurrent workers that share a single queue, so that a slow pipeline does not delay the other samples. Each sample has its own record, which links it to the corresponding trace and tracks its status. Any pipeline failure remains confined to its sample, and is recorded with the phase in which it occurred (agent call, trace collection, or metric computation). The run ends as completed, partially completed, or failed, depending on how many of its samples succeeded. A finished run can also be repeated: the new run iterates over the traces stored by the previous run and recomputes the same metrics or a new selection without repeating agent calls.

Figure 2 illustrates the pipeline divided into five sequential steps: dispatch, tracing, planning, execution, and reporting.

(1) *Dispatch*. The `AgentCallDispatcher` produces, for each sample, a `request_id` that uniquely identifies the corresponding execution and, if an agent caller is provided, calls the agent directly as a black box. All the calls are executed asynchronously, so that a batch can be evaluated concurrently to reduce the evaluation time.

(2) *Tracing*. The `TraceProcessor` uses the `request_id` to collect the trace from the observability platform, and normalizes it through a `TraceAdapter`. The format of the raw spans depends on both the provider and the agent; the adapter maps them into a canonical representation that retains the span hierarchy and timing, and enriches each span with typed semantics, such as the final answer, the ranked lists of retrieved documents, the plan snapshots, the executed steps, and the token usage of LLM calls. The normalized trace is validated and stored, so that metrics never depend on the output format of a specific agent or provider. When neither an agent caller nor an observability platform is provided, the trace is read from a JSON file, which allows SYLLO-EVAL to evaluate previously collected traces, even from different platforms.

(3) *Planning*. The `MetricPlanner` decomposes the evaluation of a sample into a sequence of `MetricPlanItems`, where each item represents the atomic computation of a specific metric on that sample. Note that, in its definition, a metric declares a *target* i.e., the span types needed to compute it (which the metric can further filter and group during computation); and the ground truths it requires, if any. Therefore, a `MetricPlanItem` contains the metric; its target in the sample under analysis, i.e., a span or a group of spans; and

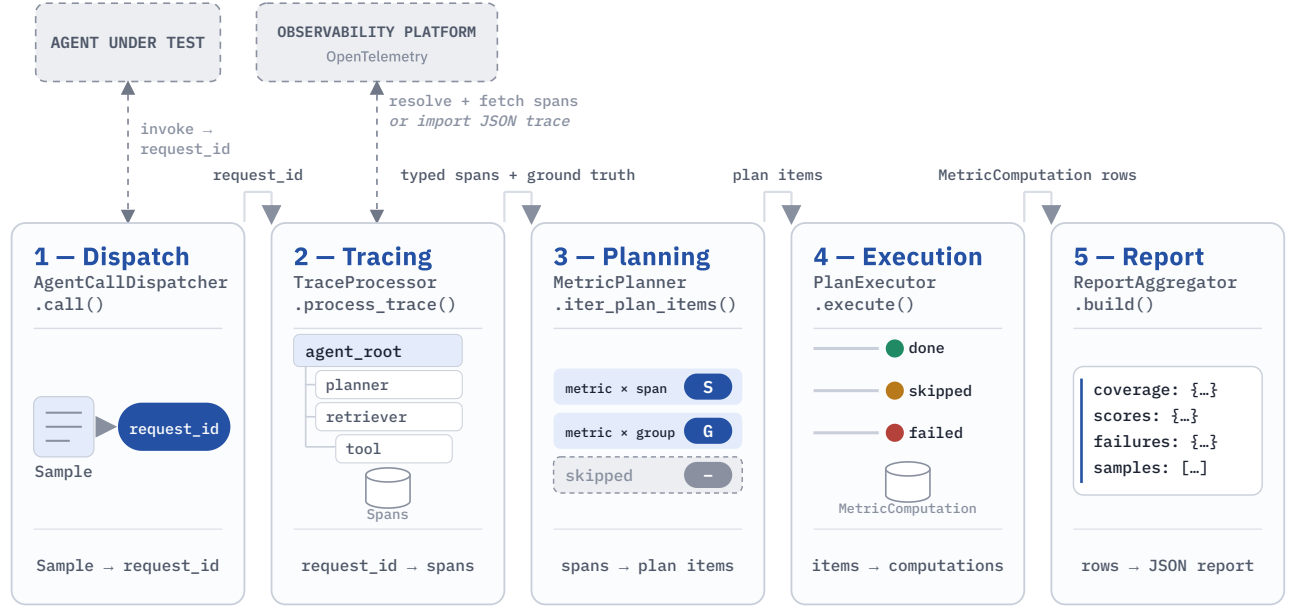


Figure 2: The SYLLO-EVAL pipeline. For each sample, the agent is invoked (1), its trace is fetched from the observability platform and normalized into typed spans (2), and the evaluation is decomposed into plan items, each pairing a metric with its target (3), which are computed concurrently (4). The resulting computations are aggregated into a JSON report (5).

the sample’s ground truths for that metric, if required. When the trace does not contain the spans targeted by a metric, or a required ground truth is missing, the planner emits a skipped item instead, together with the reason for skipping it. This way, SYLLO-EVAL never scores an incomplete observation, such as a truncated list of documents, which would bias the result.

(4) *Execution.* The `PlanExecutor` computes the plan items concurrently. For each item, it records the result, a natural-language explanation, and metadata that include the token usage and the latency of the computation, and it marks the computation as done, skipped, or failed. Since items are atomic, a failed computation is recorded with its error and does not affect the other items of the sample.

(5) *Reporting.* The `ReportAggregator` collects the computations and, on request, generates a JSON report that aggregates the results per metric and per sample. For each metric, the report lists the failures and includes a coverage field with the number of plan items that were computed, skipped, or failed, together with the reasons, so that each aggregate score is reported with the share of items on which it is computed. The report also includes the number of samples that were not processed (e.g., when a run is interrupted) and the token usage, reported separately for the agent under test and for the judges. Since not every agent records its token usage, missing values are treated as unavailable rather than as zero, which would underestimate the cost of the run, and the report states how many sources provided them.

Table 1: Built-in metrics, with the ground truth they require and the number of model calls per sample (–: deterministic). n is the number of retrieved items and C the set of claims extracted from the expected answer.

Metric	Ground truth	Calls
Plan Correctness	Expected plan	1
Plan Efficiency	Expected plan	–
Precision, Recall (set)	Relevance labels	–
NDCG@10	Relevance labels	–
Contextual Precision	Expected answer	n
Contextual Recall	Expected answer	$ C + 1$
claim-extractor variant	Expected answer	$ C + 1$
Answer Correctness	Expected answer	1

4.2 Built-in Metrics

SYLLO-EVAL includes a set of built-in metrics that can be used as the starting point of the evaluation, covering three main aspects of an agent’s execution: the plan, the retrieved context, and the final answer (Table 1). They are meant as a starting point, to be complemented by task-specific metrics and rubrics (Section 4.3).

Plan metrics. SYLLO-EVAL offers two planning metrics. *Plan Correctness* uses an LLM-as-a-Judge to assess the quality of the agent’s plan against a ground-truth plan. A judge is used because, in most agentic executions, different plans can be equally correct, so a simple comparison with the ground truth would not be flexible enough.

Since equally correct plans can still differ in efficiency, *Plan Efficiency* deterministically measures the ratio between the number of steps in the expected plan and the number of steps actually executed by the agent (including the failed ones).

Retrieval metrics. Retrieval metrics measure the agent’s capability to identify the correct items from the available knowledge bases. SYLLO-EVAL offers these metrics in label-based and LLM-based variants. When binary relevance labels are available, SYLLO-EVAL can compute deterministic *Precision*, *Recall*, and *NDCG@10*, which also takes into account the order of the retrieved context. In enterprise scenarios, however, these annotations are often not available, hence we also provide *Contextual Precision* (CP) and *Contextual Recall* (CR), which require no relevance labels.

CP uses an LLM to judge whether each retrieved item is relevant for the generation of the expected answer, rewarding the contexts in which the relevant items are ranked first:

$$CP = \frac{1}{\sum_{k=1}^n r_k} \sum_{k=1}^n r_k \cdot P@k, \quad P@k = \frac{1}{k} \sum_{i=1}^k r_i, \quad (1)$$

where n is the number of retrieved items and $r_k \in \{0, 1\}$ is the judged relevance of the item at rank k (CP = 0 if no item is relevant).

CR decomposes the expected answer into a set C of atomic claims, and uses an LLM to judge whether each claim $c \in C$ is attributable to the retrieved context ($a_c = 1$) or not ($a_c = 0$):

$$CR = \frac{1}{|C|} \sum_{c \in C} a_c. \quad (2)$$

Our implementations of CP and CR differ from their counterparts in DeepEval [18] and RAGAS [22] in how the judge is called: instead of a single call that judges all the items at once, we perform one call per item. A single judgment over a long input may be sensitive to the position of the relevant information [51]; in contrast, running a separate judgement per item ensures quality does not degrade as the retrieved context grows, in exchange for a linearly larger number of calls. To reduce the cost, our implementation relies on prefix caching.

Additionally, we include a claim-extractor variant of CR, in which claim decomposition is performed by Principled Intelligence’s CLAIM-EXTRACTOR-PRO-2605 model [74]. The claim extractor also labels the type of each claim (e.g., “Factoid”), allowing us to exclude the unverifiable claims from the score.

Answer metric. End-to-end *Answer Correctness* uses an LLM judge to compare the final answer of the agent with the expected answer. The judge focuses on semantic correctness rather than exact wording, and penalizes hallucinations and material omissions.

4.3 Customization and Deployment

New metrics, agent callers, and observability platform clients can be integrated without modifying framework’s core. Each extension implements only its role in the pipeline: an agent caller invokes the agent on a test case; a platform client collects a trace identified by a `request_id`; and a metric declares its target and required ground-truths (if any) and returns a score with an explanation and metadata. Built-in LLM-based metrics can also be customized with user-defined rubrics, allowing evaluators to encode domain-specific criteria.

Table 2: Agreement with human references (RQ1): Cohen’s κ for decisions, Spearman’s ρ for scores, AUROC against the rewards on τ^2 -bench. Human: agreement between the annotators. Abl.: SYLLO-EVAL-SC for relevance, attribution, CP, and CR; SYLLO-EVAL-WT for answers and plans. Retest: mean pairwise agreement among three runs of SYLLO-EVAL (κ or ρ).

	Human	Ours	Abl.	DeepEval	Retest
Relevance, ERB (κ)	•	•	•	•	•
Relevance, WixQA (κ)	–	•	•	•	•
Attribution, ERB (κ)	•	•	•	•	•
CP, ERB (ρ)	•	•	•	•	•
CR, ERB (ρ)	•	•	•	•	•
Answer, ERB (ρ)	•	•	•	•	•
Plan, ERB (ρ)	•	•	•	–	•
Plan, τ^2 -bench (AUROC)	–	•	•	–	•

SYLLO-EVAL can be deployed as a standalone command-line interface, as a Docker container, or as a Kubernetes pod. It can also be imported as a Python library, whose components can be used independently of one another.

5 Experimental Evaluation

We evaluate SYLLO-EVAL along three research questions:

- **RQ1:** does SYLLO-EVAL align with human evaluations?
- **RQ2:** does SYLLO-EVAL separate agents of different quality?
- **RQ3:** does SYLLO-EVAL remain accurate as retrieved contexts and traces grow?

To answer such questions, we run open-source agents on public benchmarks, collect their traces, and evaluate them with SYLLO-EVAL and other baselines. We extract judgements and metrics and we check if they match human labels; assess if the agent rankings they derive align with reference rankings; and measure how their quality and cost change evolve as inputs grow.

5.1 Experimental Setup

We employ three enterprise benchmarks:

- *EnterpriseRAG-Bench* (ERB) [86] provides a fictional company knowledge base (KB) of about 500,000 documents and 500 questions, with gold answers, atomic facts and gold documents.
- *WixQA* [17] provides a KB of 6,221 articles from the Wix Help Center and 400 customer-support queries, with gold answers and gold documents.
- τ^2 -bench [7] provides 114 customer-service tasks in an online retail store; each task has an outcome reward and a reference sequence of tool calls, which we use as the expected plan.

ERB and WixQA thus support the answer and retrieval metrics of Table 1, and τ^2 -bench the plan metrics.

On ERB and WixQA, we evaluate three open agents with diverse architectures:

- *ReAct* [99] is a low-code ReAct agent in the open-source platform Dify [44], hence it cannot be instrumented directly;
- *smolagents* [32] runs a CodeAgent, that acts by writing Python code [90] and revises an explicit plan every 3 steps;

Table 3: Scores per configuration (RQ2). ODR: Open Deep Research; f : share of search results replaced by random documents; Ans.: Answer Correctness; R: set recall; Hum.: human answer grades on the annotated questions. Bottom: Kendall’s τ with the reference order (Hum. for Ans., NDCG@10 for CP, R for CR) and with the known order of the degraded configurations; discriminative power, i.e., share of the 15 pairs of configurations that differ significantly.

Configuration	EnterpriseRAG-Bench						WixQA				
	Ans.	CP	CR	R	NDCG	Hum.	Ans.	CP	CR	R	NDCG
ReAct / Sonnet	*	*	*	*	*	*	*	*	*	*	*
ReAct / DeepSeek	*	*	*	*	*	*	*	*	*	*	*
smolagents / Sonnet	*	*	*	*	*	*	*	*	*	*	*
smolagents / DeepSeek	*	*	*	*	*	*	*	*	*	*	*
ODR / Sonnet	*	*	*	*	*	*	*	*	*	*	*
ODR / DeepSeek	*	*	*	*	*	*	*	*	*	*	*
ReAct / Sonnet, $f=0.25$	*	*	*	*	*	–	–	–	–	–	–
ReAct / Sonnet, $f=0.5$	*	*	*	*	*	–	–	–	–	–	–
Kendall’s τ , reference order	*	*	*	–	–	–	–	*	*	–	–
Kendall’s τ , known order (f)	*	*	*	*	*	–	–	–	–	–	–
Discriminative power (%)	*	*	*	*	*	*	*	*	*	*	*

- *Open Deep Research* (ODR) [42] is a multi-agent system with a supervisor delegating sub-questions to parallel researchers.

On τ^2 -bench, we use the tool-calling agent distributed with the benchmark.

We run agent with a proprietary and an open-weights model, **Claude Sonnet 5.5** and **DeepSeek-V4.1-Flash**. On ERB and WixQA this results in 6 configurations (3 agents $\times$ 2 models). On τ^2 -bench we simulate the customer with **GPT 6 Luna**, and we repeat 4 trials per run, resulting in 8 configurations (2 models $\times$ 4 trials).

We collect traces in Arize Phoenix [3], through the built-in integration of Dify or through OpenInference instrumentation [2] enabled at deployment. No agent code is modified.

We compare SYLLO-EVAL with DeepEval [18] and with two ablations, each of which removes one key design choice of Section 4:

- *DeepEval* [18] (v4.2.6, test-case mode): assesses retrieval via their own one-call implementations of CP and CR; grades answers with G-Eval [53];
- SYLLO-EVAL-SC: *single-call* ablation variant of SYLLO-EVAL that judges all retrieved items or claims in one call;
- SYLLO-EVAL-WT: *whole-trace* ablation variant of SYLLO-EVAL that reads the whole trace instead of just the target spans.

All frameworks use **Gemini 3.8 Flash** at temperature 0 as judge, chosen from a provider different from those of the agents to avoid self-preference [109]. The ablations also reuse the prompts and rubrics of SYLLO-EVAL, so that each isolates one design choice.

5.2 RQ1: Agreement with Human Judgments

We estimate the alignment between SYLLO-EVAL’s judges and humans by running the agents on the benchmarks; evaluating the resulting traces with SYLLO-EVAL; and comparing each judge-based metric with its deterministic reference, whenever the benchmark provides it. More specifically:

- *Contextual Precision*. SYLLO-EVAL judges whether each retrieved document is relevant to the expected answer. We compare these decisions with the gold documents of ERB and the expert labels of WixQA on all runs.

- *Contextual Recall*. SYLLO-EVAL judges whether each claim of the expected answer is supported by the retrieved documents. Similarly to CP, we compare these decisions with the gold documents of ERB, on its Basic and Semantic questions, which have a single gold document.
- *Answer Correctness*. SYLLO-EVAL grades the final answer against the expected one. Since no benchmark provides human grades of agent answers, we collect them on a subset of ERB and run the comparison only on that subset.
- *Plan Correctness*. SYLLO-EVAL grades the agent’s plan against the expected one. Since τ^2 -bench is the only benchmark including gold plans (i.e., the reference sequences of tool calls), we only run this comparison on τ^2 -bench.

The gold labels are reliable for positives but not for negatives. A gold document is relevant, but a non-gold document may be relevant too. Likewise, for CR we use as claims the gold facts of ERB, which are drawn from the gold document by construction [86]: a claim is supported when its gold document is retrieved, but it may also be stated in another document. For both metrics, we therefore report the share of known positives that SYLLO-EVAL accepts, and the share of the other items that it accepts, which is an upper bound on its false-positive rate.

For Answer Correctness, **two** annotators (**profile**), blind to configurations and judge outputs, grade the answers of all six configurations to **30** ERB questions, stratified by category (**180** answers), on the rubric anchors of the judge; a third annotator adjudicates disagreements. We report Spearman’s ρ between the grades of SYLLO-EVAL and the adjudicated ones, and use the agreement between the two annotators as a reference level.

For Plan Correctness, the reference is the outcome of each trajectory: we measure with AUROC how well its scores predict the outcome reward of τ^2 -bench. We also compare it with the deterministic alternative rejected in Section 4.2, i.e., matching the executed tool calls against the reference ones. The comparison matters most on the **N** successful trajectories that deviate from the reference: since the reference is only one valid solution, a plan metric should accept them, whereas matching penalizes them by construction.

To attribute differences to the judges, we also evaluate the same traces with DeepEval, where it supports the metric, and with the ablation of the corresponding design choice: SYLLO-EVAL-SC for CP and CR, and SYLLO-EVAL-WT for Answer and Plan Correctness. All frameworks receive the same inputs, i.e., the same top-10 documents and the same claims.

Finally, to separate disagreement from instability, we run each SYLLO-EVAL evaluation three times on the same stored traces (Section 4.1) and report the mean pairwise agreement between runs (retest).

[Results (Table 2): for CP and CR, accepted known positives and accepted other items; for answers, ρ vs. the inter-annotator level; for plans, AUROC, and judge vs. matching on the deviating successes; gap to the ablations and DeepEval; retest.]

5.3 RQ2: Separating Agents of Different Quality

To assess whether SYLLO-EVAL faithfully ranks which agents are better and which are worse, we fully evaluate the runs of all agents on the 480 answerable ERB questions and all WixQA queries. We rank all configurations by mean score and measure,

For each metric, we rank the six configurations by mean score and measure, with Kendall’s τ , the agreement with a reference ranking. For Answer Correctness, the reference is the ranking by human grades, and both rankings are computed on the annotated ERB questions. For the label-free CP and CR, the reference is the ranking by their label-based counterparts on the same questions, NDCG@10 and set recall, computed from the ERB gold documents and the WixQA expert labels; this tests whether an evaluator without relevance labels would rank agents as one with labels would. Since a correct ranking is of little use if differences cannot be told apart from noise, we also report the discriminative power of each metric [80], i.e., the share of the 15 configuration pairs that differ significantly (paired bootstrap over questions, Holm-corrected $p < 0.05$).

Both references are estimates: the human grades cover 30 questions, and the ERB gold documents may be incomplete [86]. Following the mock systems of ARES [79], we therefore also create configurations of known relative quality. For ReAct/Sonnet on ERB, the tool server replaces a fraction $f \in \{0.25, 0.5\}$ of the search results with random KB documents, so that $f=0 \succ f=0.25 \succ f=0.5$ by construction. We measure Kendall’s τ with this order, using R and NDCG@10 to check that the degradation took effect. Since only retrieval is degraded, the drop should appear in the retrieval metrics, and in Answer Correctness only if the agent fails to compensate, e.g., by searching again. Comparing the two shows whether the evaluation locates the degradation at its source, which an answer-only evaluation cannot do.

[Results (Table 3): τ with the reference orders; discriminative power, including that of the human grades; known order; drops at $f=0.5$, retrieval vs. answer.]

5.4 RQ3: Robustness to Input Size

RQ3 tests the premise of the framework: a judge that sees only its target should be as accurate on large inputs as on small ones, whereas a judge that reads the whole input may degrade as the

Table 4: Robustness and cost (RQ3). Small/Large: balanced accuracy at $n = 5$ and $n = 80$ for CP (WixQA) and CR (ERB); agreement in the shortest and longest tercile of trace length for answers (ERB, ρ) and plans (τ^2 -bench, AUROC). Cost per sample in the large setting: judge calls, input tokens (k), latency (s).

	Small	Large	Calls	kTok	Lat.
CP, SYLLO-EVAL	-	-	-	-	-
CP, SYLLO-EVAL-SC	-	-	-	-	-
CP, DeepEval	-	-	-	-	-
CR, SYLLO-EVAL	-	-	-	-	-
CR, SYLLO-EVAL-SC	-	-	-	-	-
CR, DeepEval	-	-	-	-	-
Answer, SYLLO-EVAL	-	-	-	-	-
Answer, SYLLO-EVAL-WT	-	-	-	-	-
Plan, SYLLO-EVAL	-	-	-	-	-
Plan, SYLLO-EVAL-WT	-	-	-	-	-

input grows [51]. We vary the size of retrieved contexts, then of traces, and finally measure the cost of each design.

For retrieved contexts, we build ranked lists of n documents, with $n \in \{5, 10, 20, 40, 80\}$, whose relevance is known by construction. For CP, we use 200 WixQA queries and hard negatives, i.e., top-ranked KB articles without an expert label (filtering criterion); for CR, 200 ERB questions with a single gold document (Basic and Semantic categories), non-gold negatives that exclude near-duplicates of the gold one (criterion), and, as claims, the gold facts that an annotator verified in the gold document. In half of the lists, the relevant item replaces the negative at the first, middle, or last rank, exposing position effects [51]; the other half contain no relevant item, testing whether a judge still finds one when there is none. We measure the balanced accuracy of the decisions as n grows. The two metrics test different things: a CP call of SYLLO-EVAL sees one document for any n , whereas each per-claim CR call still reads the whole list, so the CR experiment tests whether decomposing by claim suffices.

For traces, we compare Answer Correctness (ERB) and Plan Correctness (τ^2 -bench), which read only their target spans, with SYLLO-EVAL-WT. We split the annotated ERB runs and the τ^2 -bench trajectories into terciles of trace length, and measure in each tercile the agreement with the human grades (ρ) and with the rewards (AUROC). As trace length is confounded with agent and task, we also pad the annotated ERB traces with non-gold retrieval spans, up to 128k tokens: the padding leaves the input of SYLLO-EVAL unchanged by construction, so any change in the agreement of SYLLO-EVAL-WT is due to the added content alone.

However, per-item and per-claim calls multiply the number of judge calls (Table 1). For each framework, we report the judge calls, input tokens, and latency per sample from the run reports (Section 4.1), and the share of input tokens served from the prefix cache.

[Results (Table 4): accuracy vs. n and rank; agreement per tercile and under padding; cost and cache savings.]

6 Conclusion

We presented two tools for evaluators of enterprise agents: a novel wide-angle taxonomy of agent evaluation and SYLLO-EVAL, an open-source framework that evaluates agents from the traces they leave.

The taxonomy maps 77 papers from a fragmented field onto five questions an evaluator may ask: what is under test, how a verdict is produced, why and where agents fail, whether they are safe and compliant, and whether the evaluation is trustworthy.

The SYLLO-EVAL framework calls agents as a black box; retrieves their traces from existing observability platforms; and binds each metric to a span, group of spans, or whole trace, enforcing separation of concerns. Its built-in plan, retrieval, and answer metrics can be complemented by custom metrics and rubrics that encode domain-specific criteria, such as internal policies. On 3 open-source agents and 3 public benchmarks it agrees with human judgments ($\rho = \dots$), separates agents of different quality, and, **unlike whole-trace judges, remains stable as traces grow**.

Acknowledgments

We heartfully thank Dr Negar Arabzadeh for her insightful advice on this work.

Generative AI Disclosure

Generative AI tools were used to revise and polish the text of this manuscript and to assist with coding. The authors reviewed all generated content and take full responsibility for the content of the manuscript.

References

- [1] Waseem AlShikh, Muayad Sayed Ali, Brian Kennedy, and Dmytro Mozolevskyi. 2025. Towards Outcome-Oriented, Task-Agnostic Evaluation of AI Agents. arXiv:2511.08242. <https://arxiv.org/abs/2511.08242>
- [2] Arize AI. 2026. OpenInference: OpenTelemetry Instrumentation and Semantic Conventions for AI Observability. Accessed September 26, 2026. <https://github.com/Arize-ai/openinference>
- [3] Arize AI. 2026. Phoenix: AI Observability and Evaluation. Accessed September 26, 2026. <https://github.com/Arize-ai/phoenix>
- [4] Shir Ashury-Tahan, Yifan Mai, Elron Bandel, Michal Shmueli-Scheuer, and Leshem Choshen. 2026. ErrorMap and ErrorAtlas: Charting the Failure Landscape of Large Language Models. arXiv:2601.15812. <https://arxiv.org/abs/2601.15812>
- [5] Yarden Bakish, Amir Dudai, Roy Ganz, Oren Nuriel, Elad Ben Avraham, Mor Shpigel Nacson, and Ron Litman. 2026. Adaptive Influence Graphs for Failure Attribution in Multi-Agent Systems. arXiv:2608.24361. <https://arxiv.org/abs/2608.24361>
- [6] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. Accepted to Findings of EMNLP 2026. arXiv:2602.02475 [cs.AI] doi:10.48550/arXiv.2602.02475
- [7] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. r^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment. arXiv:2506.07982. <https://arxiv.org/abs/2506.07982>
- [8] Tara Bogavelli, Roshnee Sharma, and Hari Subramani. 2025. AgentArch: A Comprehensive Benchmark to Evaluate Agent Architectures in Enterprise. arXiv:2509.10769. <https://arxiv.org/abs/2509.10769>
- [9] Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier De Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. 2024. WorkArena+: Towards Compositional Planning and Reasoning-based Common Knowledge Work Tasks. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-0195
- [10] Fouad Bousetouane. 2026. AI Agents Do Not Fail Alone: The Context Fails First. arXiv:2607.14275. <https://arxiv.org/abs/2607.14275>
- [11] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei A. Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail?. In *Advances in Neural Information Processing Systems*, Vol. 38. doi:10.52202/085713-4082
- [12] Aditya Challapally, Chris Pease, Ramesh Raskar, and Pradyumna Chari. 2025. *The GenAI Divide: State of AI in Business 2025*. Report. MIT NANDA, MIT Media Lab. https://mlq.ai/media/quarterly_decks/v0.1_State_of_AI_in_Business_2025_Report.pdf
- [13] Ying Chang, Jiahang Xu, Xuan Feng, Chenyuan Yang, Peng Cheng, and Yuqing Yang. 2026. From Noisy Traces to Root Causes: Structural Trajectory Analysis and Causal Extraction for Agent Optimization. arXiv:2607.07702. <https://arxiv.org/abs/2607.07702>
- [14] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. 2024. Benchmarking Large Language Models in Retrieval-Augmented Generation. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 16 (2024), 17754–17762. <https://ojs.aaai.org/index.php/AAAI/article/view/29728>
- [15] Jiaju Chen, Yuxuan Lu, Xiaojie Wang, Huimin Zeng, Jing Huang, Jiri Gesi, Ying Xu, Bingsheng Yao, and Dakuo Wang. 2025. Multi-Agent-as-Judge: Aligning LLM-Agent-Based Automated Evaluation with Multi-Dimensional Human Evaluation. arXiv:2507.21028. <https://arxiv.org/abs/2507.21028>
- [16] Sangwoo Cho, Kushal Chawla, Pengshan Cai, Zefang Liu, Chenyang Zhu, Shi-Xiong Zhang, and Sambit Sahu. 2026. Ask, Don't Judge: Binary Questions for Interpretable LLM Evaluation and Self-Improvement. arXiv:2606.27226. <https://arxiv.org/abs/2606.27226>
- [17] Dvir Cohen, Lin Burg, Sviatoslav Pykhnivskiy, Hagit Gur, Stanislav Kovynov, Olga Atzmon, and Gilad Barkan. 2025. WixQA: A Multi-Dataset Benchmark for Enterprise Retrieval-Augmented Generation. arXiv:2505.08643 [cs.AI] <https://arxiv.org/abs/2505.08643>
- [18] Confident AI. 2026. DeepEval: The LLM Evaluation Framework. Accessed September 26, 2026. <https://github.com/confident-ai/deepeval>
- [19] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 37. 82895–82920. doi:10.52202/079017-2636
- [20] Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. 2025. TRAIL: Trace Reasoning and Agentic Issue Localization. arXiv:2505.08638 [cs.AI] doi:10.48550/arXiv.2505.08638
- [21] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. 2024. WorkArena: How Capable are Web Agents at Solving Common Knowledge Work Tasks?. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 11642–11662. <https://proceedings.mlr.press/v235/drouin24a.html>
- [22] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. Association for Computational Linguistics, 150–158. doi:10.18653/v1/2024.eacl-demo.16
- [23] Gartner, Inc. 2025. Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027. Press release, June 25, 2025. <https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027>
- [24] Longling Geng and Edward Y. Chang. 2025. REALM-Bench: A Benchmark for Evaluating Multi-Agent Systems on Real-world, Dynamic Planning and Scheduling Tasks. arXiv:2502.18836. <https://arxiv.org/abs/2502.18836>
- [25] Google. 2026. Google Gemini Enterprise Agent Platform Documentation. Accessed September 26, 2026. <https://docs.cloud.google.com/gemini-enterprise-agent-platform>
- [26] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. 2026. A survey on llm-as-a-judge. *The Innovation* 7, 6 (2026).
- [27] Shengyue Guan, Jindong Wang, Jiang Bian, Bin Zhu, Jian-guang Lou, and Haoyi Xiong. 2026. Evaluating LLM-based Agents for Multi-Turn Conversations: A Survey. *ACM Transactions on Intelligent Systems and Technology* 17, 4 (April 2026), 1–40. doi:10.1145/3793671
- [28] Tianyi Guan, Yiding Wang, Haotong Yang, Siyuan Cao, Shirui Liu, Yi Hu, Jiaqi Li, and Muhan Zhang. 2026. ContinualSkillBench: Can LLM Agents Truly Evolve Their Capabilities? arXiv:2608.03874. <https://arxiv.org/abs/2608.03874>
- [29] Kung-Hsiang Huang, Akshara Prabhakar, Sidharth Dhawan, Yixin Mao, Huan Wang, Silvio Savarese, Caiming Xiong, Philippe Laban, and Chien-Sheng Wu. 2025. CRMarena: Understanding the Capacity of LLM Agents to Perform Professional CRM Tasks in Realistic Environments. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 3830–3850. doi:10.18653/v1/2025.naacl-long.194
- [30] Kung-Hsiang Huang, Akshara Prabhakar, Onkar Thorat, Divyansh Agarwal, Prafulla Kumar Choubey, Yixin Mao, Silvio Savarese, Caiming Xiong, and Chien-Sheng Wu. 2026. CRMarena-Pro: Holistic Assessment of LLM Agents Across

- Diverse Business Scenarios and Interactions. *Transactions on Machine Learning Research* (Jan. 2026). <https://openreview.net/forum?id=EP1pe3Fxi1x>
- [31] Yuxuan Huang, Yihang Chen, Haozheng Zhang, Kang Li, Huichi Zhou, Meng Fang, Linyi Yang, Xiaoguang Li, Lifeng Shang, Songcen Xu, Jianye Hao, Kun Shao, and Jun Wang. 2025. Deep Research Agents: A Systematic Examination and Roadmap. *arXiv:2506.18096* [cs.AI] doi:10.48550/arXiv.2506.18096
 - [32] Hugging Face. 2025. smolagents: A Smol Library to Build Great Agentic Systems. Accessed September 29, 2026. <https://github.com/huggingface/smolagents>
 - [33] Allison Sihan Jia, Daniel Huang, Nikhil Vytla, Seung Won Wilson Yu, Nirvika Choudhury, Shayak Sen, John C. Mitchell, and Anupam Datta. 2026. What Is Your Agent's GPA? A Framework for Evaluating Agent Goal-Plan-Action Alignment. *arXiv:2510.08847* [cs.AI] <https://arxiv.org/abs/2510.08847>
 - [34] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQM66>
 - [35] Sayash Kapoor, Benedikt Strobel, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndoumga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. 2025. Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation. *arXiv:2510.11977* [cs]. doi:10.48550/arXiv.2510.11977
 - [36] NVJK Kartik, Garvit Sapra, Rishav Hada, and Nikhil Pareek. 2025. AgentCompass: Towards Reliable Evaluation of Agentic Workflows in Production. *arXiv:2509.14647* [cs.AI] doi:10.48550/arXiv.2509.14647
 - [37] Tanzila Kehkashan, Muhammad Abdullah, Ahmad Sami Al-Shamayleh, Nikola Ivković, Nor Azman Ismail, Sharifah Sakinah Syed Ahmad, Abdul Rehman, and Adnan Akhuzada. 2026. From benchmarks to deployment: a comprehensive review of agentic AI evaluation. *Artificial intelligence review* (2026).
 - [38] LangChain. 2025. State of Agent Engineering. Survey of 1,300+ professionals. Accessed September 27, 2026. <https://www.langchain.com/state-of-agent-engineering>
 - [39] LangChain. 2026. AgentEvals: Readymade Evaluators for Agent Trajectories. Accessed September 26, 2026. <https://github.com/langchain-ai/agentevals>
 - [40] LangChain. 2026. Evaluating Large Language Models With OpenEvals. Accessed September 26, 2026. <https://www.langchain.com/blog/evaluating-llms-with-openevals>
 - [41] LangChain. 2026. LangSmith Documentation. Accessed September 26, 2026. <https://docs.langchain.com/langsmith>
 - [42] LangChain. 2026. Open Deep Research. Accessed September 26, 2026. https://github.com/langchain-ai/open_deep_research
 - [43] Langfuse. 2026. Langfuse: Open Source LLM Engineering Platform. Accessed September 26, 2026. <https://github.com/langfuse/langfuse>
 - [44] LangGenius. 2026. Dify: Open-Source LLM App Development Platform. Accessed September 29, 2026. <https://github.com/langgenius/dify>
 - [45] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474.
 - [46] Haoming Li, Zhaoliang Chen, Jonathan Zhang, and Fei Liu. 2025. PLANET: A Collection of Benchmarks for Evaluating LLMs' Planning Capabilities. *arXiv:2504.14773*. <https://arxiv.org/abs/2504.14773>
 - [47] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods. *arXiv:2412.05579*. <https://arxiv.org/abs/2412.05579>
 - [48] Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Tianhao Wu, Banghua Zhu, Joseph E. Gonzalez, and Ion Stoica. 2024. From Crowdsourced Data to High-Quality Benchmarks: Arena-Hard and BenchBuilder Pipeline. *arXiv:2406.11939*. <https://arxiv.org/abs/2406.11939>
 - [49] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher Ré, Diana Acosta-Navas, Drew A. Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue Wang, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research* (Aug. 2023). <https://openreview.net/forum?id=iO4LZibEqW>
 - [50] Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tianwei Yue, Chi Wang, Jian Kang, Qingyun Wu, and Huazheng Wang. 2026. Who&When Pro: Can LLMs Really Attribute Failures in AI Agents? *arXiv:2607.09996* [cs.AI] doi:10.48550/arXiv.2607.09996
 - [51] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tacl_a_00638
 - [52] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *The Twelfth International Conference on Learning Representations*. <https://arxiv.org/abs/2308.03688>
 - [53] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2511–2522. doi:10.18653/v1/2023.emnlp-main.153
 - [54] Dario Loi, Elena Maria Muià, Federico Siciliano, Giovanni Trappolini, Vincenzo Crisà, Peter Kruger, and Fabrizio Silvestri. 2025. AutoBench: Automating LLM Evaluation through Reciprocal Peer Assessment. *arXiv:2510.22593*. <https://arxiv.org/abs/2510.22593>
 - [55] Pengrui Lu, Bingyu Xu, Wenjun Zhang, Shengjia Hua, Xuanjian Gao, Ranshiang Ge, Lyumanshan Ye, Linxuan Wu, Yiran Li, Junfei Fish Yu, et al. 2026. AlphaEval: Evaluating Agents in Production. *arXiv:2604.12162* [cs.CL] doi:10.48550/arXiv.2604.12162
 - [56] Xing Han Lü, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. *arXiv:2504.08942*. <https://arxiv.org/abs/2504.08942>
 - [57] Riccardo Lunardi, Vincenzo Della Mea, Stefano Mizzaro, and Kevin Roitero. 2025. On Robustness and Reliability of Benchmark-Based Evaluation of LLMs. *arXiv:2509.04013*. <https://arxiv.org/abs/2509.04013>
 - [58] Hanjun Luo, Shenyu Dai, Chiming Ni, Xinfeng Li, Guibin Zhang, Kun Wang, Tongliang Liu, and Hanan Salam. 2025. AgentAuditor: Human-Level Safety and Security Evaluation for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 38. <https://openreview.net/forum?id=2KKq7MWJM>
 - [59] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujia Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 37. 74325–74362. doi:10.52202/079017-2365
 - [60] Netta Madvil, Gilad Dym, Alon Mcelati, Edo Dekel, Jonathan Liberman, Rotem Brazilay, Liron Schliesser, Max Svidlo, Shai Nir, Orel Shalom, Yaron Friedman, David Connack, Amos Rimon, Philip Tannor, and Shir Chorev. 2026. Holistic Evaluation and Failure Diagnosis of AI Agents. *arXiv:2605.14865* [cs.AI] doi:10.48550/arXiv.2605.14865
 - [61] Kiana Jafari Meimandi, Gabriela Aránguiz-Dias, Grace Ra Kim, Lana Saadeddin, Allie Griffith, and Mykel J. Kochenderfer. 2025. The Measurement Imbalance in Agentic AI Evaluation Undermines Industry Productivity Claims. *arXiv:2506.02064*. <https://arxiv.org/abs/2506.02064>
 - [62] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: a benchmark for General AI Assistants. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=fbxvavhs3>
 - [63] Microsoft. 2026. Microsoft Copilot Studio Documentation. Accessed September 26, 2026. <https://learn.microsoft.com/en-us/microsoft-copilot-studio/>
 - [64] MLflow Project. 2026. MLflow Tracing for LLM Observability. Accessed September 26, 2026. <https://mlflow.org/docs/latest/genai/tracing/>
 - [65] Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. 2025. Evaluation and Benchmarking of LLM Agents: A Survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*. ACM, 6129–6139. doi:10.1145/3711896.3736570
 - [66] Hadar Mulian, Sergey Zeltyn, Ido Levy, Liane Galanti, Avi Yaeli, and Segev Shlomov. 2026. AgentFixer: From Failure Detection to Fix Recommendations in Agentic Systems. In *Proceedings of the 2026 International Workshop on Agentic Engineering*. ACM, 96–103. <https://dl.acm.org/doi/10.1145/3786167.3788427>
 - [67] Vishal Ishwar Naik, Chenyu Xu, Donna Dong, Hussein Hassan, Abhishek Pradhan, Ofer Mendelevitch, Tallat Shafat, and Humayun Irshad. 2026. Guardian-AgentBench: Where Agents Fail and How to Guard Them. *arXiv:2607.20982*. <https://arxiv.org/abs/2607.20982>
 - [68] OpenAI. 2025. Introducing Deep Research. Published February 2, 2025. Accessed September 26, 2026. <https://openai.com/index/introducing-deep-research/>
 - [69] OpenTelemetry Authors. 2026. Semantic Conventions for Generative AI Systems. Status: Development. Accessed September 26, 2026. <https://github.com/open-telemetry/semantic-conventions-genai>
 - [70] Melissa Z. Pan, Negar Arabzadeh, Riccardo Cogo, Yuxuan Zhu, Alexander Xiong, Lakshya A. Agrawal, Huanzhi Mao, Emma Shen, Sid Pallerla, Liana Patel, Shu Liu, Tianneng Shi, Xiaoyuan Liu, Jared Quincy Davis, Emmanuele Lacavalla, Alessandro Basile, Shuyi Yang, Paul Castro, Daniel Kang, Koushik Sen, Dawn Song, Joseph E. Gonzalez, Ion Stoica, Matei Zaharia, and Marquita Ellis. 2026.

- Measuring Agents in Production. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research)*. PMLR. <https://openreview.net/forum?id=mWxEAgz3xu>
- [71] Priyaranjan Pattanayak, Amit Agarwal, Bhargava Kumar, Yeshil Bangera, Srikant Panda, Tejaswini Kumar, and Hitesh Laxmichand Patel. 2025. Review of Reference Generation Methods in Large Language Models. *International Journal of Artificial Intelligence & Machine Learning* 4, 1 (2025), 21–42. https://iaeme.com/MasterAdmin/Journal_uploads/IJAIML/VOLUME_4_ISSUE_1/IJAIML_04_01_003.pdf
 - [72] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Mailard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. 2021. KILT: a Benchmark for Knowledge Intensive Language Tasks. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2523–2544. <https://aclanthology.org/2021.naacl-main.200>
 - [73] Nearchos Potamitis, Vansh Ramani, Har Ashish Arora, Dhairya Kuchhal, Lars Klein, and Akhil Arora. 2025. ReasonBENCH: Benchmarking the (In)Stability of LLM Reasoning. arXiv:2512.07795. <https://arxiv.org/abs/2512.07795>
 - [74] Principled Intelligence. 2026. Orbitals: LLM Guardrails Tailored to Your Principles. Accessed September 26, 2026. <https://github.com/Principled-Intelligence/orbitals>
 - [75] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihao Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=dHng200Jjr>
 - [76] Md Nakhla Rafi, Md Ahasanuzzaman, Dong Jae Kim, Zhijie Wang, and Tse-Hsun Chen. 2026. FALAT: Tracing Failures in LLM Agent Trajectories via Dependency-Guided Search. arXiv:2606.00765. <https://arxiv.org/abs/2606.00765>
 - [77] Ragas. 2026. Ragas metrics: Agentic or Tool use. Accessed September 26, 2026. https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/agents/
 - [78] Selvan Sunitha Ravi, Bartosz Mielczarek, Anand Kannappan, Douwe Kiela, and Rebecca Qian. 2024. Lynx: An Open Source Hallucination Evaluation Model. arXiv:2407.08488. <https://arxiv.org/abs/2407.08488>
 - [79] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 338–354. doi:10.18653/v1/2024.naacl-long.20
 - [80] Tetsuya Sakai. 2006. Evaluating Evaluation Metrics Based on the Bootstrap. In *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '06)*. ACM, 525–532. doi:10.1145/1148170.1148261
 - [81] Alireza Salemi and Hamed Zamani. 2024. Evaluating Retrieval Quality in Retrieval-Augmented Generation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2395–2400. doi:10.1145/3626772.3657957
 - [82] Divyanshu Saxena, Rishikesh Maurya, Xiaoxuan Ou, Gagan Somashekar, Shachee Mishra Gupta, Arun Iyer, Yu Kang, Chetan Bansal, Aditya Akella, and Saravan Rajmohan. 2025. Continuous Benchmark Generation for Evaluating Enterprise-scale LLM Agents. arXiv:2511.10049. <https://arxiv.org/abs/2511.10049>
 - [83] Shreya Shankar, J. D. Zamfirescu-Pereira, Björn Hartmann, Aditya G. Parameswaran, and Ian Arawjo. 2024. Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. ACM, Article 131, 14 pages. doi:10.1145/3654777.3676450
 - [84] Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. 2026. Harness-R1: Learning to Edit Executable Runtime Harnesses from Agent Failure Trajectories. arXiv:2608.02276. <https://arxiv.org/abs/2608.02276>
 - [85] Reshabh K. Sharma, Shraddha Barke, and Benjamin Zorn. 2026. Willful Disobedience: Automatically Detecting Failures in Agentic Traces. In *Proceedings of the ACM Conference on AI and Agentic Systems*. 584–596. arXiv:2603.23806. <https://arxiv.org/abs/2603.23806>
 - [86] Yuhong Sun, Joachim Rahmfeld, Chris Weaver, Weijia Chen, Roshan Desai, Wenxi Huang, and Mark H. Butler. 2026. EnterpriseRAG-Bench: A RAG Benchmark for Company Internal Knowledge. arXiv:2605.05253 [cs.LR] doi:10.48550/arXiv.2605.05253
 - [87] Giovanni Trappolini, Florin Cuconasu, Simone Filice, Yoelle Maarek, and Fabrizio Silvestri. 2025. Redefining Retrieval Evaluation in the Era of LLMs. arXiv:2510.21440. <https://arxiv.org/abs/2510.21440>
 - [88] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 16022–16076. doi:10.18653/v1/2024.acl-long.850
 - [89] Harsh Vishwakarma, Ankush Agarwal, Ojas Patil, Chaitanya Devaguptapu, and Mahesh Chandran. 2025. Can LLMs Help You at Work? A Sandbox for Evaluating LLM Agents in Enterprise Environments. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 9167–9201. doi:10.18653/v1/2025.emnlp-main.466
 - [90] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR.
 - [91] Yawen Wang, Wenjie Wu, Junjie Wang, and Qing Wang. 2026. From flat logs to causal graphs: Hierarchical failure attribution for llm-based multi-agent systems. arXiv preprint arXiv:2602.23701 (2026).
 - [92] Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. 2026. Rethinking the Evaluation of Harness Evolution for Agents. arXiv:2607.12227. <https://arxiv.org/abs/2607.12227>
 - [93] Zhaohui Geoffrey Wang. 2026. AgentTrace: Causal Graph Tracing for Root Cause Analysis in Deployed Multi-Agent Systems. arXiv:2603.14688. <https://arxiv.org/abs/2603.14688>
 - [94] Alva West, Yixuan Weng, Minjun Zhu, Zhen Lin, Zhiyuan Ning, and Yue Zhang. 2025. Abduct, Act, Predict: Scaffolding Causal Inference for Automated Failure Attribution in Multi-Agent Systems. arXiv:2509.10401 [cs.AI] <https://arxiv.org/abs/2509.10401>
 - [95] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-1650
 - [96] Frank F Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Zora Z Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Melroy Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. 2024. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. arXiv:2412.14161. <https://arxiv.org/abs/2412.14161>
 - [97] Shu-Xun Yang, Cunxiang Wang, Haoke Zhang, Wenbo Yu, Lindong Wu, Jiayi Gui, Dayong Yang, Yukuo Cen, Zhuoer Feng, Bosi Wen, Yidong Wang, Lucen Zhong, Jiamin Ren, Linfeng Zhang, and Jie Tang. 2026. TraceSIR: A Multi-Agent Framework for Structured Analysis and Reporting of Agentic Execution Traces. arXiv:2603.00623. <https://arxiv.org/abs/2603.00623>
 - [98] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=roNSXZpUDN>
 - [99] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.
 - [100] Samuel Yeh, Yiwen Zhu, Shaleen Deep, and Sharon Li. 2026. Tracing Agentic Failure from the Flow of Success. arXiv:2607.12747. <https://arxiv.org/abs/2607.12747>
 - [101] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2026. A Survey on Evaluation of LLM-based Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 26690–26714. doi:10.18653/v1/2026.findings-acl.1330
 - [102] Runyang Yu, Hongru Cai, Caiqi Zhang, Qiancheng Xu, Meng Liu, Tiezheng Yu, Yongqi Li, and Wenjie Li. 2026. A Survey on Agent-as-a-Judge. arXiv:2601.05111 <https://arxiv.org/abs/2601.05111>
 - [103] Fangyi Yu. 2025. When AIs Judge AIs: The Rise of Agent-as-a-Judge Evaluation for LLMs. arXiv:2508.02994. <https://arxiv.org/abs/2508.02994>
 - [104] Bang Zhang, Ruotian Ma, Qingxuan Jiang, Peisong Wang, Jiaqi Chen, Zheng Xie, Xingyu Chen, Yue Wang, Fanghua Ye, Jian Li, Yifan Yang, Zhaopeng Tu, and Xiaolong Li. 2025. Sentient Agent as a Judge: Evaluating Higher-Order Social Cognition in Large Language Models. arXiv:2505.02847. <https://arxiv.org/abs/2505.02847>
 - [105] Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2026. AgenTracer: Who Is Inducing Failure in the LLM Agentic Systems?. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=I05Dseqvud>
 - [106] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2025. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. arXiv:2510.04618. <https://arxiv.org/abs/2510.04618>

- [107] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 76583–76599. <https://proceedings.mlr.press/v267/zhang25cq.html>
- [108] Zhixin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. 2024. Agent-SafetyBench: Evaluating the Safety of LLM Agents. arXiv:2412.14470. <https://arxiv.org/abs/2412.14470>
- [109] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, Vol. 36, 46595–46623. doi:10.52202/075280-2020
- [110] Ming Zhong, Yang Liu, Da Yin, Yuning Mao, Yizhu Jiao, Pengfei Liu, Chenguang Zhu, Heng Ji, and Jiawei Han. 2022. Towards a Unified Multi-Dimensional Evaluator for Text Generation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023–2038. doi:10.18653/v1/2022.emnlp-main.131
- [111] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=oKn9c6ytLx>
- [112] Jiachen Zhu, Menghui Zhu, Renting Rui, Rong Shan, Congmin Zheng, Bo Chen, Yunjia Xi, Jianghao Lin, Weiwen Liu, Ruiming Tang, Yong Yu, and Weinan Zhang. 2025. Evolutionary Perspectives on the Evaluation of LLM-Based AI Agents: A Comprehensive Survey. arXiv:2506.11102 [cs.CL]. doi:10.48550/arXiv.2506.11102
- [113] Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM Agents Fail and How They can Learn From Failures. arXiv:2509.25370 <https://arxiv.org/abs/2509.25370>
- [114] Kunlun Zhu, Yifan Luo, Dingling Xu, Yukun Yan, Zhenghao Liu, Shi Yu, Ruobing Wang, Shuo Wang, Yishan Li, Nan Zhang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. RAGEval: Scenario Specific RAG Evaluation Dataset Generation Framework. arXiv:2408.01262. <https://arxiv.org/abs/2408.01262>
- [115] Mingchen Zhuge, Changsheng Zhao, Dylan R. Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. 2025. Agent-as-a-Judge: Evaluate Agents with Agents. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 80569–80611. <https://proceedings.mlr.press/v267/zhuge25a.html>

Received 28 September 2026; revised 28 September 2026; accepted 28 September 2026